

# Backpropagation, Part 2: How It Runs in Code

## Contents

|                                                 |    |
|-------------------------------------------------|----|
| 1. Overview . . . . .                           | 2  |
| 2. Prerequisites . . . . .                      | 3  |
| 3. The regression algorithm, in loops . . . . . | 3  |
| 4. The regression code . . . . .                | 3  |
| 4.1 Initialising and predicting . . . . .       | 3  |
| 4.2 Updating the parameters . . . . .           | 4  |
| 4.3 Training . . . . .                          | 4  |
| 5. The same training in Keras . . . . .         | 5  |
| 6. The classification problem . . . . .         | 6  |
| 7. The classification derivatives . . . . .     | 6  |
| 7.1 The output layer . . . . .                  | 6  |
| 7.2 The hidden layer . . . . .                  | 8  |
| 7.3 Student 1 in numbers . . . . .              | 8  |
| 8. Training the classifier . . . . .            | 9  |
| 9. Summary . . . . .                            | 10 |
| 10. Key terms . . . . .                         | 10 |

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



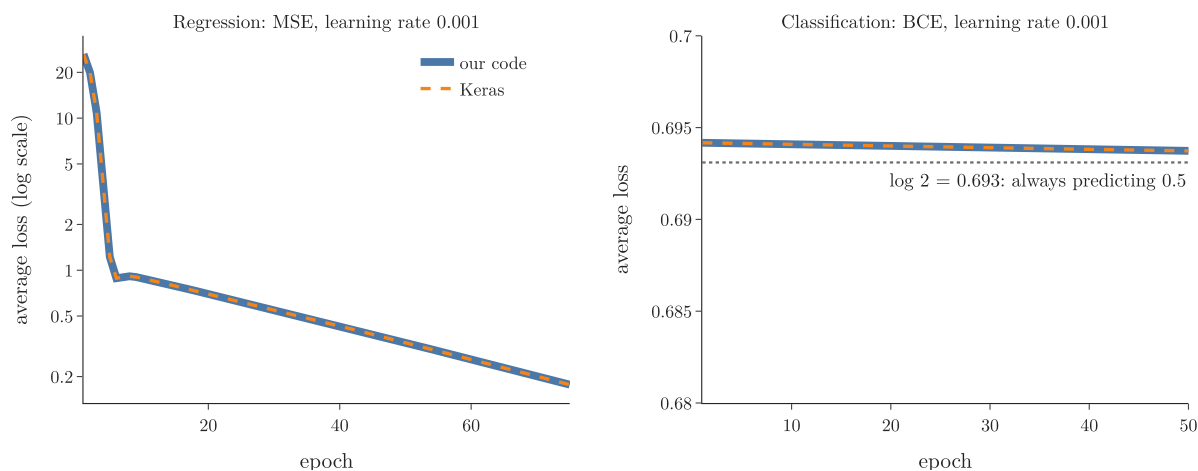
- **Builds on:** Gradient descent ([Note 57](#)); Sigmoid function ([Note 74](#)); Derivatives of one variable ([Note 600](#)); Partial derivatives and gradients ([Note 601](#)); Forward propagation ([Note 1010](#)); Loss functions in deep learning ([Note 1014](#)).
- **Leads to:** Vanishing gradient ([Note 1018](#)); Improving a neural network ([Note 1021](#)).

## 1. Overview

**Key point:** We turn the algorithm into code twice: for regression (linear nodes, squared error) and for classification (sigmoid nodes, binary cross-entropy). Our code and Keras, started from the same weights, produce the same losses to six digits.

The [backpropagation what Note](#) built the algorithm and its 9 derivatives for a 2-2-1 regression network. This Note runs it:

1. **Regression:** the same network and data, trained for several epochs with our own NumPy code, then with Keras.
2. **Classification:** the same architecture with sigmoid activations and binary cross-entropy. The derivatives change, so we derive them, then train again.



**Figure 1:** Average loss per epoch for our code and for Keras, started from the same weights

Figure 1 shows the result: in both problems the two lines lie on top of each other. Keras runs the same backpropagation we write by hand.

## 2. Prerequisites

- The [backpropagation what Note](#): the algorithm, the network and the regression derivatives.
- The [sigmoid derivative Note](#) and the [logistic regression gradient Note](#), for the classification derivatives.

## 3. The regression algorithm, in loops

**Key point:** An outer loop over epochs, an inner loop over the 4 students; inside: forward, loss, update all 9 parameters. At the end of each epoch, average the 4 losses.

The data and network are those of the [backpropagation what Note](#): four students (CGPA, profile score, package), a 2-2-1 network with linear nodes, all weights 0.1 and biases 0. The algorithm:

1. Initialise the parameters.
2. For each of 5 epochs:
  - for each student, in order:
    - a. predict  $\hat{y}$  with forward propagation;
    - b. compute the loss  $(y - \hat{y})^2$ ;
    - c. update the 9 parameters with  $W \leftarrow W - \eta \partial L / \partial W$ ,  $\eta = 0.001$ .
  - print the average of the 4 losses: the loss of the epoch.

Students are taken in order here. In practice the rows are usually picked in a random order each epoch, which the [gradient descent in neural networks Note](#) returns to.

## 4. The regression code

**Key point:** Four small functions: initialise, forward, update, train. The update function is the 9 derivative formulas, one line per group.

### 4.1 Initialising and predicting

**Key point:** One weight matrix and one bias vector per layer; forward propagation keeps the hidden outputs, because the update needs them.

**Python:** Initialisation and forward propagation.

```
import numpy as np

def initialize_parameters(layer_dims):
    # weights 0.1, biases 0; W[l] has shape (nodes in l-1, nodes in l)
    return {l: [np.full((layer_dims[l-1], layer_dims[l]), 0.1),
                 np.zeros(layer_dims[l])]
            for l in range(1, len(layer_dims))}
```

```
def forward_regression(x, params):
    a1 = params[1][0].T @ x + params[1][1]      # 011, 012
    y_hat = params[2][0].T @ a1 + params[2][1]  # 021
    return y_hat[0], a1
```

`initialize_parameters([2, 2, 1])` creates a  $2 \times 2$  matrix  $W^1$ , a bias vector of length 2, a  $2 \times 1$  matrix  $W^2$  and one bias: the 9 parameters. Each layer is the matrix product of the [forward propagation Note](#), without the sigmoid.

For student 1 (8, 8) the function returns  $\hat{y} = 0.32$  and the hidden outputs  $a^1 = (1.6, 1.6)$ . We keep  $a^1$  because the derivatives of the output weights are  $-2(y - \hat{y}) O_{11}$  and  $-2(y - \hat{y}) O_{12}$ .

## 4.2 Updating the parameters

**Key point:** Compute  $g = -2(y - \hat{y})$  once; every update is  $g$  times the parameter's own factors.

**Python:** The 9 updates.

```
def update_regression(params, x, y, y_hat, a1, lr):
    g = -2 * (y - y_hat)                # dL/dy_hat
    W2 = params[2][0][:, 0].copy()      # keep the old W2
    params[2][0][:, 0] -= lr * g * a1    # W2_11, W2_21
    params[2][1][0] -= lr * g           # b21
    params[1][0] -= lr * np.outer(x, g * W2) # four W1
    params[1][1] -= lr * g * W2         # b11, b12
```

`-=` changes the arrays in place, so `params` holds the new values afterwards. The copy of `W2` matters: the hidden-layer derivatives use the weights from before this update.

The minus signs in the formulas and the minus of the update rule cancel: every update here adds  $\eta \cdot 2(y - \hat{y})$  times the parameter's factors.

## 4.3 Training

**Key point:** Five epochs bring the average loss from 26.35 down to 1.22.

**Python:** The training loop.

```
def train_regression(X, Y, epochs=5, lr=0.001):
    params, history = initialize_parameters([2, 2, 1]), []
    for epoch in range(epochs):
        losses = []
        for x, y in zip(X, Y):          # one student at a time
            y_hat, a1 = forward_regression(x, params)
            losses.append((y - y_hat) ** 2)
            update_regression(params, x, y, y_hat, a1, lr)
        history.append(np.mean(losses))  # loss of the epoch
```

```
return params, history
```

The first epoch is exactly the one worked by hand in the [backpropagation what Note](#): losses 13.54, 21.29, 30.43 and 40.12, average 26.35. Then:

| Epoch        | 1     | 2     | 3     | 4    | 5    |
|--------------|-------|-------|-------|------|------|
| Average loss | 26.35 | 19.86 | 10.88 | 3.78 | 1.22 |

After 5 epochs the weights have moved from 0.1 to  $W_{11}^1 = W_{12}^1 = 0.272$ ,  $W_{21}^1 = W_{22}^1 = 0.393$ ,  $W_{11}^2 = W_{21}^2 = 0.469$ , with biases 0.028 (hidden) and 0.122 (output). The predictions are 5.14, 5.25, 5.36 and 5.84 LPA for packages of 4, 5, 6 and 7. They are in the right range and rising in the right order, but still too close together. After 75 epochs the average loss is 0.177 (Figure 1, left).

## 5. The same training in Keras

**Key point:** Same architecture, same starting weights, plain SGD with the same learning rate, one row per update, no shuffling: Keras gives the same losses and weights.

**Python:** The regression network in Keras, with our starting weights.

```
import keras

model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(2, activation="linear"),
    keras.layers.Dense(1, activation="linear"),
])
model.set_weights([np.full((2, 2), 0.1), np.zeros(2),
                  np.full((2, 1), 0.1), np.zeros(1)])
model.compile(optimizer=keras.optimizers.SGD(learning_rate=0.001),
              loss="mse")
model.fit(X, Y, epochs=5, batch_size=1, shuffle=False)
model.summary() shows 9 trainable parameters. Keras normally starts from random
weights; get_weights returns the arrays in the order kernel, bias, kernel, bias, and
set_weights overwrites them. batch_size=1 updates after every row, as our loop does;
shuffle=False keeps the rows in order.
```

Keras prints the epoch losses 26.346, 19.859, 10.882, 3.777 and 1.224, and ends with exactly our weights (0.272, 0.393, 0.469, ...). The two curves in Figure 1 (left) coincide for all 75 epochs.

**Extra:** Every setting must match to get identical numbers. By default `fit` shuffles the rows every epoch and uses batches of 32 rows, so with default settings Keras follows a different path and ends at a slightly different loss. With an optimizer such as Adam

instead of plain SGD, the steps themselves change.

## 6. The classification problem

**Key point:** Same architecture, but every node uses a sigmoid and the loss is binary cross-entropy. Two extra steps appear in forward propagation: each weighted sum goes through the sigmoid.

Now the output is whether the student is placed (1) or not (0):

| Student | CGPA | Profile score | Placed |
|---------|------|---------------|--------|
| 1       | 8    | 8             | 1      |
| 2       | 7    | 9             | 1      |
| 3       | 6    | 10            | 0      |
| 4       | 5    | 5             | 0      |

Two things change:

- **Activations:** every node computes its weighted sum  $z$  and then outputs  $\sigma(z)$ , as in the [forward propagation Note](#).
- **Loss:** [binary cross-entropy](#),  $L = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$ , the right loss for two classes with a sigmoid output (see the [loss functions Note](#)).

The algorithm (loops, forward, loss, update) stays the same. Only the derivatives change.

For student 1, with all weights 0.1 and biases 0:

$$z_{11} = 0.1 \times 8 + 0.1 \times 8 = 1.6, \quad O_{11} = O_{12} = \sigma(1.6) = 0.832$$

$$z_f = 0.1 \times 0.832 + 0.1 \times 0.832 = 0.166, \quad \hat{y} = \sigma(0.166) = 0.5415, \quad L = -\log 0.5415 = 0.613$$

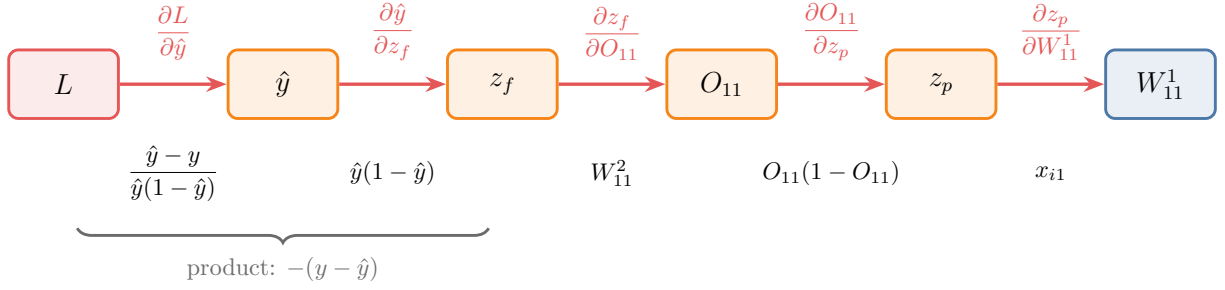
## 7. The classification derivatives

**Key point:** The sigmoid and the log cancel:  $\partial L / \partial z_f = -(y - \hat{y})$ . The output layer then looks like regression with  $-1$  in place of  $-2$ ; the hidden layer gains a factor  $O(1 - O)$  from its own sigmoid.

### 7.1 The output layer

**Key point:** Three links,  $L \rightarrow \hat{y} \rightarrow z_f \rightarrow W_{11}^2$ ; the first two multiply to  $-(y - \hat{y})$ .

gradient travels from the loss back to the weight; each arrow multiplies by its local derivative



**Figure 2:** The chain from the loss to a first-layer weight in the classification network: each arrow multiplies by its local derivative

Call  $z_f = W_{11}^2 O_{11} + W_{21}^2 O_{12} + b_{21}$  the weighted sum of the output node, so  $\hat{y} = \sigma(z_f)$ . A change in  $W_{11}^2$  changes  $z_f$ , then  $\hat{y}$ , then  $L$ :

$$\frac{\partial L}{\partial W_{11}^2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_f} \cdot \frac{\partial z_f}{\partial W_{11}^2}$$

The first two factors (Figure 2, left):

- Differentiating the loss:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

- The sigmoid's derivative (see the [sigmoid derivative Note](#)):  $\partial \hat{y} / \partial z_f = \hat{y}(1 - \hat{y})$ .

Multiplied, the  $\hat{y}(1 - \hat{y})$  cancels, the same simplification as in section 4.3 of the [logistic regression gradient Note](#):

$$\frac{\partial L}{\partial z_f} = \hat{y} - y = -(y - \hat{y})$$

The last factor is the same as in regression:  $\partial z_f / \partial W_{11}^2 = O_{11}$ ,  $\partial z_f / \partial W_{21}^2 = O_{12}$ ,  $\partial z_f / \partial b_{21} = 1$ . So:

$$\frac{\partial L}{\partial W_{11}^2} = -(y - \hat{y}) O_{11}, \quad \frac{\partial L}{\partial W_{21}^2} = -(y - \hat{y}) O_{12}, \quad \frac{\partial L}{\partial b_{21}} = -(y - \hat{y})$$

These are the regression formulas with  $-1$  in place of  $-2$ .

## 7.2 The hidden layer

**Key point:** Two more links:  $z_f \rightarrow O_{11}$  gives  $W_{11}^2$ ;  $O_{11} \rightarrow z_p$  gives the sigmoid derivative  $O_{11}(1 - O_{11})$ .

For  $W_{11}^1$  the chain is five links long (Figure 2). With  $z_p = W_{11}^1 x_{i1} + W_{21}^1 x_{i2} + b_{11}$  the weighted sum of hidden node 1, so  $O_{11} = \sigma(z_p)$ :

$$\frac{\partial L}{\partial W_{11}^1} = \underbrace{\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_f}}_{-(y - \hat{y})} \cdot \underbrace{\frac{\partial z_f}{\partial O_{11}}}_{W_{11}^2} \cdot \underbrace{\frac{\partial O_{11}}{\partial z_p}}_{O_{11}(1 - O_{11})} \cdot \underbrace{\frac{\partial z_p}{\partial W_{11}^1}}_{x_{i1}}$$

The other five follow the same pattern. Only the last factor changes within a node ( $x_{i1}$ ,  $x_{i2}$  or 1), and the second node uses  $W_{21}^2$  and  $O_{12}$ :

| Parameter                    | $\partial L / \partial (\text{parameter})$                            |
|------------------------------|-----------------------------------------------------------------------|
| $W_{11}^1, W_{21}^1, b_{11}$ | $-(y - \hat{y}) W_{11}^2 O_{11}(1 - O_{11}) \times x_{i1}, x_{i2}, 1$ |
| $W_{12}^1, W_{22}^1, b_{12}$ | $-(y - \hat{y}) W_{21}^2 O_{12}(1 - O_{12}) \times x_{i1}, x_{i2}, 1$ |

Compared with regression, each hidden derivative has one extra factor,  $O(1 - O)$ : the derivative of the hidden node's own sigmoid.

## 7.3 Student 1 in numbers

**Key point:**  $-(y - \hat{y}) = -0.4585$ ; output weights get  $-0.381$ , hidden weights  $-0.0513$ .

1. **In words:** compute  $-(y - \hat{y})$  once, then multiply by each parameter's factors.
2. **Formula:** the formulas of Sections 7.1 and 7.2.
3. **Example:**  $y = 1$ ,  $\hat{y} = 0.5415$ ,  $O_{11} = 0.832$ :

$$-(y - \hat{y}) = -(1 - 0.5415) = -0.4585$$

$$\frac{\partial L}{\partial W_{11}^2} = -0.4585 \times 0.832 = -0.3815, \quad \frac{\partial L}{\partial b_{21}} = -0.4585$$

$$\frac{\partial L}{\partial W_{11}^1} = -0.4585 \times 0.1 \times 0.832 \times 0.168 \times 8 = -0.0513, \quad \frac{\partial L}{\partial b_{11}} = -0.0064$$

The hidden-layer gradients are about 7 times smaller than the output-layer ones: the factors  $W_{11}^2 = 0.1$  and  $O_{11}(1 - O_{11}) = 0.14$  shrink them. Each sigmoid between a weight and the loss multiplies its gradient by at most 0.25, an effect the [vanishing gradient Note](#) studies.

**Python:** The classification gradients.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```



```
def gradients_classification(params, x, y, y_hat, a1):
    d_out = -(y - y_hat) # dL/dz_f
    W2 = params[2][0][:, 0]
    d_hidden = d_out * W2 * a1 * (1 - a1) # dL/dz_p, per node
    return {"W2": d_out * a1, "b2": d_out,
            "W1": np.outer(x, d_hidden), "b1": d_hidden}
```

`d_hidden` holds, for each hidden node, the derivative of the loss with respect to that node's weighted sum. Every weight entering the node multiplies it by its own input. In the Notebook, `tf.GradientTape` returns the same 9 numbers.

## 8. Training the classifier

**Key point:** The loss starts at 0.694 and is still 0.694 after 50 epochs. Keras, from the same start, does exactly the same: the code is right, and this network is stuck.

Training with the same loops and  $\eta = 0.001$ :

| Epoch    | 1      | 2      | 50     |
|----------|--------|--------|--------|
| Our code | 0.6942 | 0.6942 | 0.6937 |
| Keras    | 0.6942 | 0.6942 | 0.6937 |

The loss barely moves, and every student gets a probability of about 0.54. Is the code wrong? The same network in Keras (sigmoid activations, `loss="binary_crossentropy"`, same weights and settings) gives the same numbers to four digits (Figure 1, right). So backpropagation is computed correctly; the network is simply not learning.

The loss sits next to  $\log 2 = 0.693$ , the binary cross-entropy of predicting 0.5 for everyone: the network has learned nothing beyond “two students are placed, two are not”. Several causes add up:

- **A tiny learning rate** with tiny hidden gradients (Section 7.3): the weights hardly move.
- **Identical hidden nodes:** every weight starts at 0.1, so both hidden nodes stay the same, and the network acts as if it had one hidden node.
- **Four rows:** very little data to learn from.

**Extra:** With  $\eta = 0.1$  and 2,000 epochs the loss falls to 0.49: students 1 and 2 get 0.66 and student 3 gets 0.01, but student 4 still gets 0.66 instead of a low value. The two hidden nodes are still equal, so the network still acts as if it had a single hidden node. With  $\eta = 0.5$  the steps are too big and the loss ends higher, at 0.76. Starting weights, learning rate and the sigmoid's small slope are exactly the problems that the Notes on improving a network address.

## 9. Summary

|                                         | Regression                        | Classification                                       |
|-----------------------------------------|-----------------------------------|------------------------------------------------------|
| Activations                             | linear everywhere                 | sigmoid everywhere                                   |
| Loss                                    | $(y - \hat{y})^2$                 | $-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$        |
| $\partial L / \partial z$ at the output | $-2(y - \hat{y})$                 | $-(y - \hat{y})$                                     |
| Output-layer gradient                   | $-2(y - \hat{y}) O_{1j}$          | $-(y - \hat{y}) O_{1j}$                              |
| Hidden-layer gradient                   | $-2(y - \hat{y}) W_{j1}^2 x_{ik}$ | $-(y - \hat{y}) W_{j1}^2 O_{1j} (1 - O_{1j}) x_{ik}$ |
| Training here                           | 26.35 to 1.22 in 5 epochs         | stuck at 0.694                                       |

- The code is the algorithm: initialise, then epochs of forward, loss, update, row by row.
- Our NumPy code and Keras (`SGD`, `batch_size=1`, `shuffle=False`, same starting weights) give identical losses and weights.
- For a sigmoid output with binary cross-entropy,  $\partial L / \partial z = \hat{y} - y$ .
- Each sigmoid on the way back adds a factor  $O(1 - O)$ , which shrinks the gradient.
- Correct backpropagation can still fail to learn: starting weights, learning rate and data size matter.

## 10. Key terms

| Term                                   | Meaning                                                                                                                         |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>set_weights / get_weights</code> | Keras methods to write and read a model's weights and biases, as a list of arrays                                               |
| <code>batch_size=1</code>              | Keras setting that updates the weights after every single row                                                                   |
| <code>shuffle=False</code>             | Keras setting that keeps the rows in their original order in every epoch                                                        |
| <code>keras.optimizers.SGD</code>      | Plain gradient descent in Keras, with a fixed learning rate                                                                     |
| $\partial L / \partial z$              | The derivative of the loss with respect to a node's weighted sum; every weight entering the node multiplies it by its own input |